

+++ date = '2026-04-30T01:04:30-07:00' draft = false title = 'Practica3: Paradigma Funcional' +++

Universidad Autonoma de Baja California

Materia: Paradigmas de la programacion

Docente: Jose Carlos Gallegos Mariscal

Alumno: Carlos David Cruz Santiesteban

Matricula: 379909

Practica 3: Paradigma funcional

Introducción

Para esta practica utilizaremos el lenguaje de programación haskell. Instalamos y utilizaremos todas las funciones que nos ofrece su pagina web, desde el compilador hasta su interprete.

Para descargarlo e implementarlo es bastante sencillo, la pagina de haskell te ofrece un script que puedes ejecutar directamente en la linea de comando, esto instalara los siguientes elementos:

- GHCup → Herramienta de instalación e entorno de desarrollo, el enlace ejecutado descarga y realiza la instalación utilizando esta herramienta.
- GHC → Compilador de Haskell
- Hugs → Interprete interactivo de Haskell.
- HLS → Haskell Language Server, este no lo utilizan directamente. contiene las librerías estándar y código de funcionamiento de Haskell, del cual hacen uso GHC y Hugs.
- Stack → Manejador de paquetes, similar a Pip en Python o apt en Ubuntu/Debian.
- Cabal → Herramienta de empaquetado de binarios (buildtool), se encarga de utilizar Stack para descargar dependencias y GHC para compilar el código en un solo comando.

Hello World en Haskell

Para probar las herramientas instaladas, ejecutamos algunos programas sencillos

```
main = do
  putStrLn "Hello, world!"
  putStrLn ("Please look at my favorite odd numbers: " ++ show (filter
odd[10..20]))
```

Una impresion en pantalla sencilla, imprime "Hello, world!" seguido de una serie de numeros del 11 al 19

```
> 6 + 3^2 * 4
42

> take 10 (filter even [43..])
[44,46,48,50,52,54,56,58,60,62]
```

Ejecutamos las anteriores lineas de codigo con el interprete de haskell (ghci), las lineas que comienzan con '>' son las instrucciones que le damos al interprete, y las siguientes son los resultados de dichas instrucciones.

Aplicación to do en haskell

Para hacer una aplicacion ToDo en haskell, lo haremos con Stack. Lo primero que haremos es configurar el entorno de desarrollo.

- Usaremos el comando: `$stack new <project>`
- Se mostrara texto en la consola, finalizando con un mensaje de confirmacion
- Creara archivos viendose algo similar a esto

app	Include code for blog post	5 years ago
src	Include code for blog post	5 years ago
test	Include code for blog post	5 years ago
.gitignore	Include code for blog post	5 years ago
ChangeLog.md	Include code for blog post	5 years ago
LICENSE	Include code for blog post	5 years ago
README.md	Include code for blog post	5 years ago
Setup.hs	Include code for blog post	5 years ago
package.yaml	Include code for blog post	5 years ago
stack.yaml	Include code for blog post	5 years ago
stack.yaml.lock	Include code for blog post	5 years ago
todo-exe	Include code for blog post	5 years ago
todo.cabal	Include code for blog post	5 years ago

- Dentro del archivo package.yaml veremos algo como

```
dependencies:
- base >= 4.7 && < 5
```

- Debemos agregar `dotenv` y el paquete `open-browser`

```
dependencies:
- base >= 4.7 && < 5
- dotenv
- open-browser
```

- Utilizaremos el comando `$stack test` y se mostrara:

```
$stack test
Registering library for todo-0.1.0.0..
todo> test (suite: todo-test)

Test suite not yet implemented

todo> Test suite todo-test passed
Completed 2 action(s).
```

- Para ver las compilaciones de tu proyecto Haskell usa `$stack run` y mostrar el resultado en tu consola

```
$stack run
someFunc
```

Codigo fuente

En esta parte agregaremos informacion en el main.hs que esta en la carpeta app, y el Lib.hs que esta en src
Nuestro main.hs debe verse algo asi:

```
module Main where

import Lib (prompt)

main :: IO ()
main = do
    putStrLn "Commands:"
    putStrLn "+ <String> - Add a TODO entry"
    putStrLn "- <Int>    - Delete the numbered entry"
    putStrLn "s <Int>    - Show the numbered entry"
    putStrLn "e <Int>    - Edit the numbered entry"
    putStrLn "l          - List todo"
    putStrLn "r          - Reverse todo"
    putStrLn "c          - Clear todo"
    putStrLn "q          - Quit"
    prompt [] -- Start with the empty todo list.
```

Y el lib.hs debe quedar asi:

```
module Lib
  ( prompt,
    editIndex,
  )
where

import Data.List
```

```

putTodo :: (Int, String) -> IO ()
putTodo (n, todo) = putStrLn (show n ++ ": " ++ todo)

prompt :: [String] -> IO ()
prompt todos = do
  putStrLn ""
  putStrLn "Usa +(create), -(delete), s(how), e(dit), l(ist), r(everse), c(lear),
q(uit).\"
  command <- getLine
  if "e" `isPrefixOf` command
    then do
      print "¿Cuál es el nuevo texto para esa tarea?"
      newTodo <- getLine
      editTodo command todos newTodo
    else interpret command todos

interpret :: String -> [String] -> IO ()
interpret ('+' : ' ' : todo) todos = prompt (todo : todos)
interpret ('-' : ' ' : num) todos =
  case deleteOne (read num) todos of
    Nothing -> do
      putStrLn "No existe una tarea con ese número"
      prompt todos
    Just todos' -> prompt todos'
interpret ('s' : ' ' : num) todos =
  case showOne (read num) todos of
    Nothing -> do
      putStrLn "No existe una tarea con ese número"
      prompt todos
    Just todo -> do
      print $ num ++ ". " ++ todo
      prompt todos
interpret "l" todos = do
  print $ show (length todos) ++ " en total"
  mapM_ putTodo (zip [0..] todos)
  prompt todos
interpret "r" todos = do
  let reversed = reverseTodos todos
  mapM_ putTodo (zip [0..] reversed)
  prompt todos
interpret "c" _ = do
  print "Lista limpiada."
  prompt []
interpret "q" _ = return ()
interpret command todos = do
  putStrLn ("Comando inválido: `" ++ command ++ "`")
  prompt todos

-- Funciones auxiliares

deleteOne :: Int -> [a] -> Maybe [a]
deleteOne 0 (_ : as) = Just as
deleteOne n (a : as) = do
  as' <- deleteOne (n - 1) as

```

```

    return (a : as')
deleteOne _ [] = Nothing

showOne :: Int -> [a] -> Maybe a
showOne n todos =
    if (n < 0) || (n > length todos)
    then Nothing
    else Just (todos !! n)

editIndex :: Int -> a -> [a] -> [a]
editIndex i x xs = take i xs ++ [x] ++ drop (i + 1) xs

editTodo :: String -> [String] -> String -> IO ()
editTodo ('e' : ' ' : num) todos newTodo =
    case editOne (read num) todos newTodo of
        Nothing -> do
            putStrLn "No existe una tarea con ese número"
            prompt todos
        Just old -> do
            print $ "Anterior: " ++ old
            print $ "Nuevo:    " ++ newTodo
            let newTodos = editIndex (read num :: Int) newTodo todos
            mapM_ putTodo (zip [0..] newTodos)
            prompt newTodos

editOne :: Int -> [a] -> String -> Maybe a
editOne n todos _ =
    if (n < 0) || (n > length todos)
    then Nothing
    else Just (todos !! n)

reverseTodos :: [a] -> [a]
reverseTodos xs = go xs []
    where
        go [] ys = ys
        go (x : xs) ys = go xs (x : ys)

```

También actualizaremos el `speech.hs` que está en test, lo actualizaremos para que las funciones de la librería funcionen correctamente.

```

import Control.Exception
import Lib (editIndex)

main :: IO ()
main = do
    putStrLn "Test:"
    let index      = 1
    let new_todo   = "dos"
    let todos      = ["Escribir", "un", "blog", "post"]
    let expected   = ["Escribir", "dos", "blog", "post"]

```

```
let result    = editIndex index new_todo todos == expected
putStrLn $ assert result "editIndex funcionó correctamente."
```

Compilar

Para compilar el proyecto utilizamos el comando `stack build`. La primera vez puede tardar varios minutos.

Para ejecutar la app usamos `stack run`

Deberia de verse en pantalla algo como esto:

```
Commands:
+ <String> - Add a TODO entry
- <Int>     - Delete the numbered entry
s <Int>     - Show the numbered entry
e <Int>     - Edit the numbered entry
l           - List todo
r           - Reverse todo
c           - Clear todo
q           - Quit
```

Ahora corremos los tests con `stack test`. Si todo es correcto en pantalla debe salir

```
Test:
editIndex funcionó correctamente.
todo> Test suite todo-test passed
```

Conclusión

Hasta el momento haskell y en general el paradigma funcional, es el lenguaje mas difícil con el que he trabajado. Es completamente diferente con los demás paradigmas, en las anteriores practicas, habia similitudes entre la programacion estructurada y la orientacion a objetos. Sin embargo es bastante interesante el funcionamiento y las posibles aplicaciones que tiene haskell.